

Quantium analysis task 2 Tushar

Tushar_K

2026-06-24

Experimentation & uplift testing (Quantium Task 2)

This is the solution file for Task 2 on the lines of quantium virtual internship.

Step 1: Understanding the problem statement

Task is to evaluate the stores' sales performance with a comparison among controlled vs trial group involving store 77, 86 and 88 and providing recommendation for these locations based on the result findings.

Step 2: Preparing data

We have been given the data file in csv format named "QVI_data.csv". I am going to use R for this analysis.

Now moving the files in desired directory for entire R analysis and installing and loading required packages.

```
# setwd("C:/Users/Admin/Downloads")
```

```
# Installing and loading required packages (I have masked them for rmd file)
```

```
# Installing and Loading required packages
```

```
#install.packages("tidyverse")
```

```
#install.packages("tidyr")
```

```
#install.packages("ggplot2")
```

```
#install.packages("readr")
#install.packages("data.table")
#install.packages("knitr")
#install.packages("devtools")
#devtools::install_github("r-lib/conflicted")
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidyr)
library(ggplot2)
library(readr)
library(data.table)
```

```
##
## Attaching package: 'data.table'
##
## The following objects are masked from 'package:lubridate':
##
##   hour, isoweek, isoyear, mday, minute, month, quarter, second, wday,
##   week, yday, year
##
## The following objects are masked from 'package:dplyr':
##
##   between, first, last
##
```

```
## The following object is masked from 'package:purrr':  
##  
##      transpose  
##  
## The following object is masked from 'package:base':  
##  
##      %notin%
```

```
library(knitr)  
library(conflicted)  
library(lubridate)  
library(dplyr)
```

```
# default to the dplyr version of filter  
conflicts_prefer(dplyr::filter)
```

```
## [conflicted] Will prefer dplyr::filter over any other package.
```

```
conflicts_prefer(dplyr::lag)
```

```
## [conflicted] Will prefer dplyr::lag over any other package.
```

```
# csv file  
file.path <- ("C:/Users/Admin/Downloads/")  
csv_data <- read_csv(paste0(file.path, "QVI_data.csv"))  
  
# view dataset if it is loaded perfectly and what it contains  
head(csv_data)
```

Load and Read the raw data file

```
##      LYLTY_CARD_NBR      DATE STORE_NBR TXN_ID PROD_NBR
```

```
##          <int>      <IDat>      <int> <int>      <int>
## 1:      1000 2018-10-17          1    1          5
## 2:      1002 2018-09-16          1    2         58
## 3:      1003 2019-03-07          1    3         52
## 4:      1003 2019-03-08          1    4        106
## 5:      1004 2018-11-02          1    5         96
## 6:      1005 2018-12-28          1    6         86
##
##          PROD_NAME PROD_QTY TOT_SALES PACK_SIZE
##          <char>      <int>      <num>      <int>
## 1: Natural Chip      Compny SeaSalt175g          2          6.0          175
## 2: Red Rock Deli Chikn&Garlic Aioli 150g          1          2.7          150
## 3: Grain Waves Sour      Cream&Chives 210G          1          3.6          210
## 4: Natural ChipCo      Hony Soy Chckn175g          1          3.0          175
## 5:      WW Original Stacked Chips 160g          1          1.9          160
## 6:      Cheetos Puffs 165g          1          2.8          165
##
##          BRAND      LIFESTAGE PREMIUM_CUSTOMER
##          <char>      <char>      <char>
## 1: NATURAL YOUNG SINGLES/COUPLES      Premium
## 2: RRD YOUNG SINGLES/COUPLES      Mainstream
## 3: GRNWVES YOUNG FAMILIES      Budget
## 4: NATURAL YOUNG FAMILIES      Budget
## 5: WOOLWORTHS OLDER SINGLES/COUPLES      Mainstream
## 6: CHEETOS MIDAGE SINGLES/COUPLES      Mainstream
```

```
# {view(csv_data)} (in case i want to see the full picture)
```

Setting themes for plots Giving plots a crisp white background and adjusting plot themes at middle of plot header

```
theme_set(theme_bw())
theme_update(plot.title = element_text(hjust = 0.5))
```

Selecting control stores

Store 77, 86, 88 are trial stores.

I need to match trial stores to control stores that were similar in metrics before the trial period started in February 2019. The metrics that I considered for it are as follows:

- 1) Monthly sales revenue
- 2) Monthly customer footfall
- 3) Monthly transactions per customer

In order to select stores some precautions need to be kept in mind, these are:

- 1) The stores must be operational throughout the trial period for a quality study.
- 2)

```
# adding monthID_y_m column
csv_data[, monthID_y_m := format(DATE, "%Y_%m")]

head(csv_data)
```

```
##      LYLTY_CARD_NBR      DATE STORE_NBR TXN_ID PROD_NBR
##      <int>      <IDat>      <int>  <int>  <int>
## 1:      1000 2018-10-17          1      1      5
## 2:      1002 2018-09-16          1      2     58
## 3:      1003 2019-03-07          1      3     52
## 4:      1003 2019-03-08          1      4    106
## 5:      1004 2018-11-02          1      5     96
## 6:      1005 2018-12-28          1      6     86
##
##              PROD_NAME PROD_QTY TOT_SALES PACK_SIZE
##              <char>      <int>      <num>      <int>
## 1: Natural Chip      Compny SeaSalt175g          2        6.0        175
## 2: Red Rock Deli Chikn&Garlic Aioli 150g          1        2.7        150
## 3: Grain Waves Sour   Cream&Chives 210G          1        3.6        210
## 4: Natural ChipCo     Hony Soy Chckn175g          1        3.0        175
## 5:      WW Original Stacked Chips 160g          1        1.9        160
## 6:      Cheetos Puffs 165g          1        2.8        165
##
##      BRAND      LIFESTAGE PREMIUM_CUSTOMER monthID_y_m
##      <char>      <char>      <char>      <char>
## 1:  NATURAL  YOUNG SINGLES/COUPLES      Premium    2018_10
## 2:    RRD    YOUNG SINGLES/COUPLES      Mainstream  2018_09
## 3:  GRNWVES      YOUNG FAMILIES      Budget      2019_03
## 4:  NATURAL      YOUNG FAMILIES      Budget      2019_03
## 5: WOOLWORTHS  OLDER SINGLES/COUPLES      Mainstream  2018_11
## 6:  CHEETOS  MIDAGE SINGLES/COUPLES      Mainstream  2018_12
```

```
library(dplyr)

store_monthly_metrics <- csv_data %>%
  group_by(STORE_NBR, monthID_y_m) %>%
  summarise(
    total_sales = sum(TOT_SALES, na.rm = TRUE),
    num_customers = n_distinct(LYLT_Y_CARD_NBR),
    txn_per_cust = n_distinct(TXN_ID) / num_customers,
    chips_per_cust = sum(PROD_QTY, na.rm = TRUE) / num_customers,
    avg_price_per_unit = total_sales / sum(PROD_QTY, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(STORE_NBR, monthID_y_m)

head(store_monthly_metrics)
```

```
## # A tibble: 6 x 7
##   STORE_NBR monthID_y_m total_sales num_customers txn_per_cust chips_per_cust
##       <int> <chr>         <dbl>         <int>         <dbl>         <dbl>
## 1         1 2018_07           207.             49           1.06           1.27
## 2         1 2018_08           176.             42           1.02           1.29
## 3         1 2018_09           279.             59           1.05           1.27
## 4         1 2018_10           188.             44           1.02           1.32
## 5         1 2018_11           193.             46           1.02           1.24
## 6         1 2018_12           190.             42           1.12           1.36
## # i 1 more variable: avg_price_per_unit <dbl>
```

```
library(dplyr)

# Filter to the pre-trial months (Before February 2019)
pre_trial_data <- store_monthly_metrics %>%
  filter(monthID_y_m < "2019_02")

# Identify and keep stores with exactly 7 months of data
pre_trial_stores_7month <- pre_trial_data %>%
  group_by(STORE_NBR) %>%
  filter(n() == 7) %>%
```

```

ungroup()

head(pre_trial_stores_7month)

## # A tibble: 6 x 7
##   STORE_NBR monthID_y_m total_sales num_customers txn_per_cust chips_per_cust
##   <int> <chr>         <dbl>         <int>         <dbl>         <dbl>
## 1         1 2018_07           207.           49           1.06           1.27
## 2         1 2018_08           176.           42           1.02           1.29
## 3         1 2018_09           279.           59           1.05           1.27
## 4         1 2018_10           188.           44           1.02           1.32
## 5         1 2018_11           193.           46           1.02           1.24
## 6         1 2018_12           190.           42           1.12           1.36
## # i 1 more variable: avg_price_per_unit <dbl>

```

Now I am ranking potential control stores' based on similarity to the trial stores

```

calculateCorrelation <- function(inputTable, metricCol, trial_store) {

  calcCorrTable <- data.frame(Store1 = integer(),
                              Store2 = integer(),
                              corr_measure = numeric())

  storeNumbers <- unique(inputTable$STORE_NBR)

  # Extracting the trial store's data and ensure it is ordered chronologically
  trial_data <- inputTable[inputTable$STORE_NBR == trial_store, ]
  trial_data <- trial_data[order(trial_data$monthID_y_m), ]
  trial_metric <- trial_data[[metricCol]] # Extract just the column we want to measure

  # Loop through every store to compare it against the trial store
  for (i in storeNumbers) {

    # Extract the potential control store's data and order it chronologically
    control_data <- inputTable[inputTable$STORE_NBR == i, ]

```

```

control_data <- control_data[order(control_data$monthID_y_m), ]
control_metric <- control_data[[metricCol]]

# Calculate the Pearson correlation between the two stores
# 'complete.obs' handles any NA values safely
correlation_value <- cor(trial_metric, control_metric, use = "complete.obs")

# Store the result in a temporary row, then attach it to our main results table
temp_row <- data.frame(Store1 = trial_store,
                      Store2 = i,
                      corr_measure = correlation_value)

calcCorrTable <- rbind(calcCorrTable, temp_row)
}

# Return the final table of correlations
return(calcCorrTable)
}

```

Checking correlations for the stores individually and Visually plotting for finding similarity in stores' patterns:

Store number 77

```

# Calculate correlation for total sales for Trial Store 77
corr_nSales <- calculateCorrelation(inputTable = pre_trial_stores_7month,
                                   metricCol = "total_sales",
                                   trial_store = 77)

# top 5 correlated stores
head(corr_nSales[order(-corr_nSales$corr_measure), ])

```

##	Store1	Store2	corr_measure
## 73	77	77	1.0000000
## 68	77	71	0.9141060
## 222	77	233	0.9037742
## 112	77	119	0.8676644
## 16	77	17	0.8426684

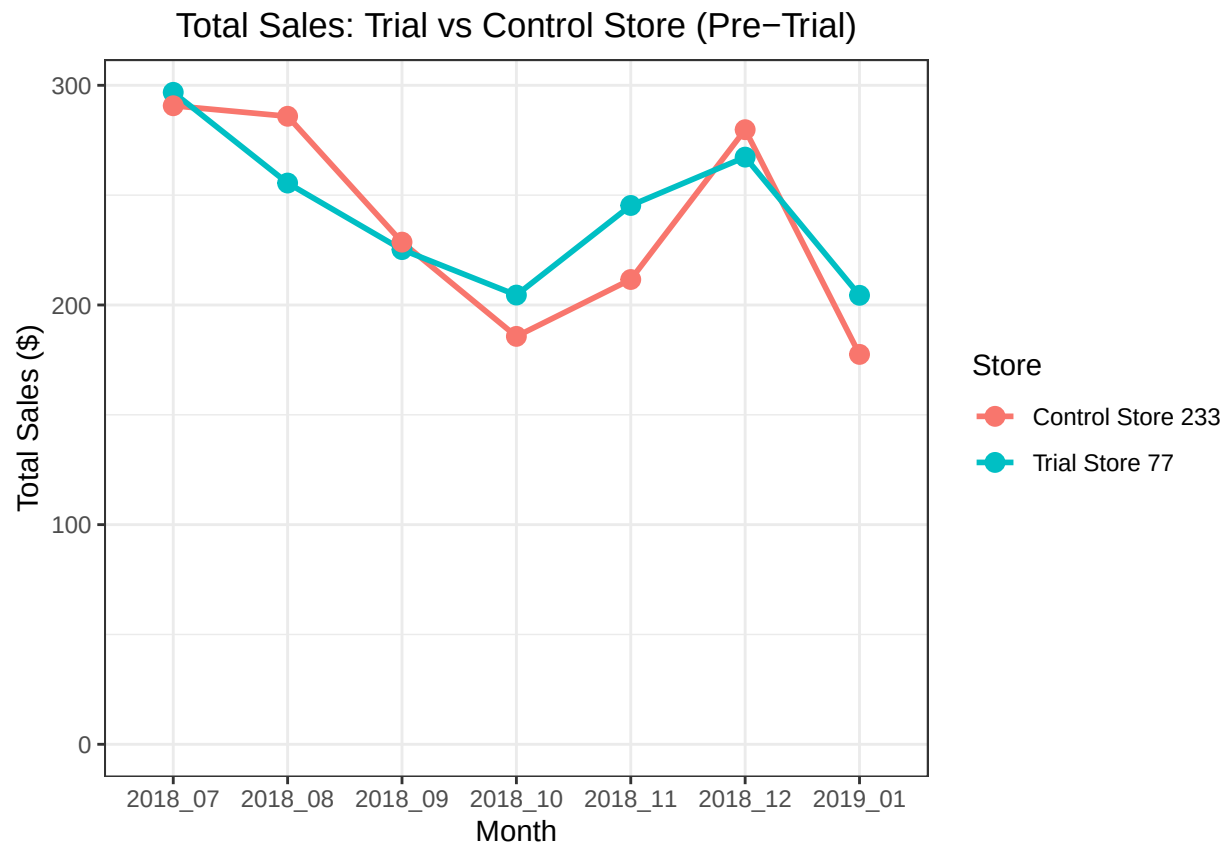
```
## 3      77      3      0.8066436
```

```
library(ggplot2)
library(dplyr)

# Plotting FOR STORE 77-----

# Define your stores (I can review the plots and their correlations by just changing the control_store number right below)
trial_store <- 77
control_store <- 233 #(I tried 71,233 , 119, 17, 3 as well but 233 showed consistent results)
# Filter the pre-trial dataset for just these two stores
plot_data <- pre_trial_stores_7month %>%
  filter(STORE_NBR %in% c(trial_store, control_store)) %>%
  # Create a clearer label for the legend
  mutate(Store_Type = ifelse(STORE_NBR == trial_store,
                             paste("Trial Store", trial_store),
                             paste("Control Store", control_store)))

# Generate the line chart
ggplot(plot_data, aes(x = monthID_y_m, y = total_sales, color = Store_Type, group = Store_Type)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "Total Sales: Trial vs Control Store (Pre-Trial)",
       x = "Month",
       y = "Total Sales ($)",
       color = "Store") +
  # Expand the y-axis to start at 0 to prevent visual distortion of the gap
  scale_y_continuous(limits = c(0, NA))
```



Store number 86

```
# Calculate correlation for total sales for Trial Store 86
corr_nSales <- calculateCorrelation(inputTable = pre_trial_stores_7month,
                                   metricCol = "total_sales",
                                   trial_store = 86)

# top 5 correlated stores
head(corr_nSales[order(-corr_nSales$corr_measure), ])
```

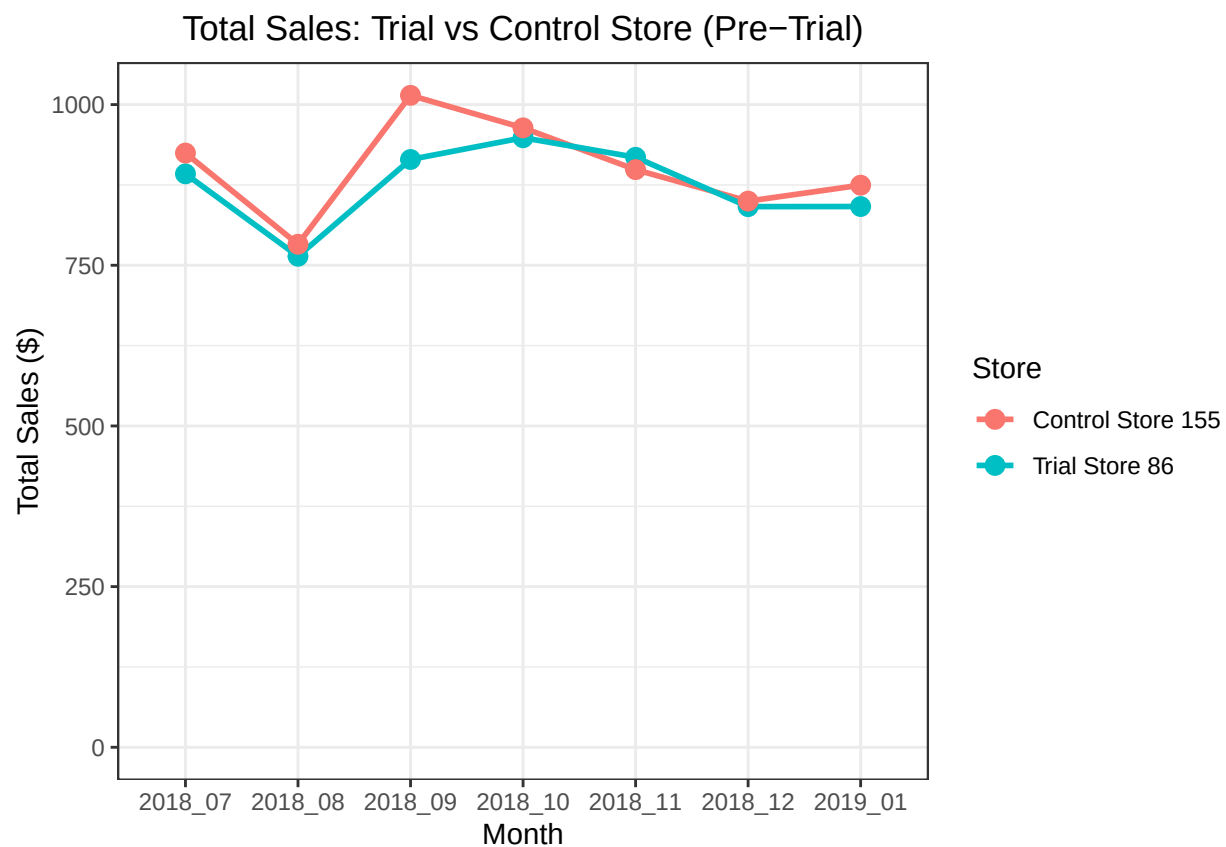
```
##      Store1 Store2 corr_measure
## 81      86      86      1.0000000
## 148     86     155      0.8778817
## 125     86     132      0.8465166
## 229     86     240      0.8250658
## 211     86     222      0.7950753
## 103     86     109      0.7882995
```

```
library(ggplot2)
library(dplyr)

# Plotting for STORE 86-----

# Define your stores (I can review the plots and their correlations by just changing the control_store number right below)
trial_store <- 86
control_store <- 155 # (I tried 155, 132, 240, 222, 109 but 155 showed best results, although 222 and 109 were also close enough)
# Filter the pre-trial dataset for just these two stores
plot_data <- pre_trial_stores_7month %>%
  filter(STORE_NBR %in% c(trial_store, control_store)) %>%
  # Create a clearer label for the legend
  mutate(Store_Type = ifelse(STORE_NBR == trial_store,
                             paste("Trial Store", trial_store),
                             paste("Control Store", control_store)))

# Generate the line chart
ggplot(plot_data, aes(x = monthID_y_m, y = total_sales, color = Store_Type, group = Store_Type)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "Total Sales: Trial vs Control Store (Pre-Trial)",
       x = "Month",
       y = "Total Sales ($)",
       color = "Store") +
  # Expand the y-axis to start at 0 to prevent visual distortion of the gap
  scale_y_continuous(limits = c(0, NA))
```



Store number 88

```
# Calculate correlation for total sales for Trial Store 88
corr_nSales <- calculateCorrelation(inputTable = pre_trial_stores_7month,
                                   metricCol = "total_sales",
                                   trial_store = 88)

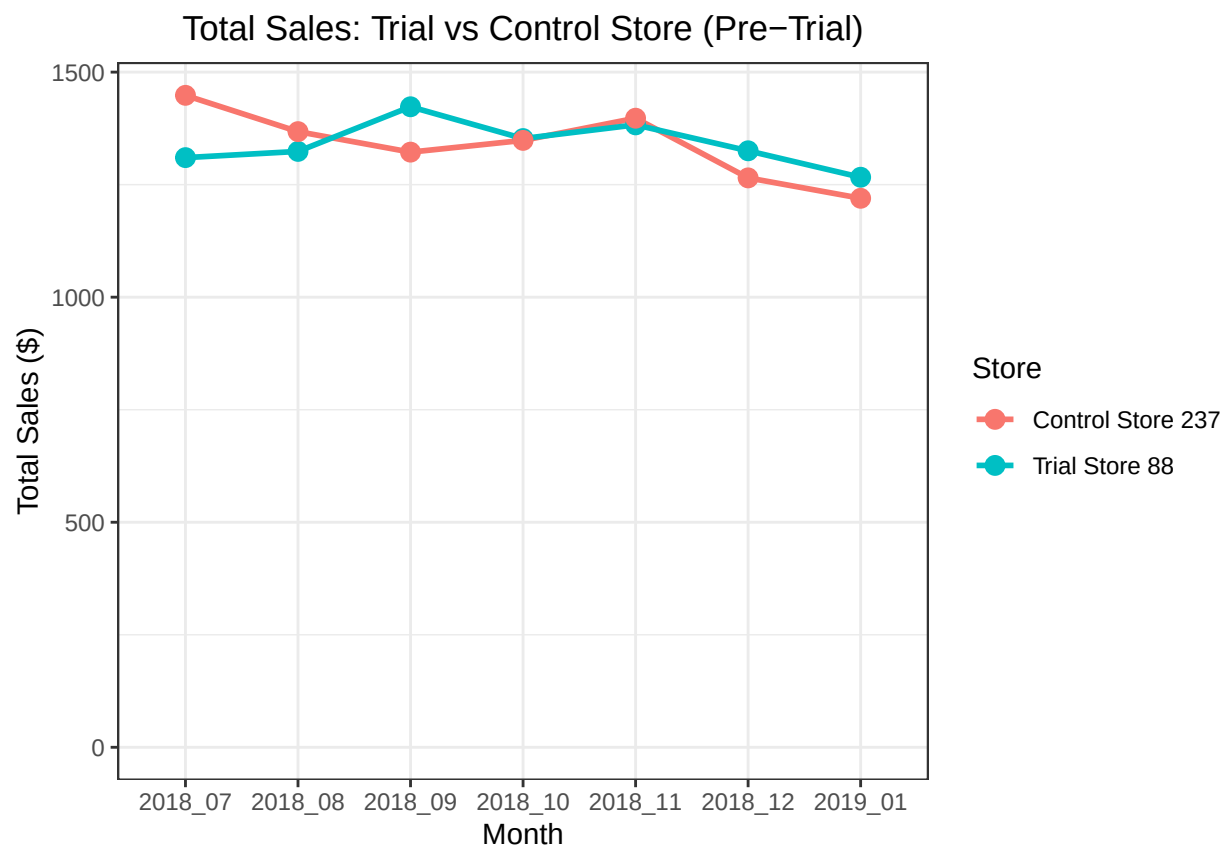
# top 5 correlated stores
view(corr_nSales[order(-corr_nSales$corr_measure), ])
#for this store the top5 didn't show that much similarity so I added another metric.
```

```

# Plotting for STORE 88-----
library(ggplot2)
library(dplyr)
# Define your stores (I can review the plots and their correlations by just changing the control_store number right below)
trial_store <- 88
control_store <- 237 #(I tried 159,237, 204, 134, 1, 253 and 237 showed best results)
# Filter the pre-trial dataset for just these two stores
plot_data <- pre_trial_stores_7month %>%
  filter(STORE_NBR %in% c(trial_store, control_store)) %>%
  # Create a clearer label for the legend
  mutate(Store_Type = ifelse(STORE_NBR == trial_store,
                             paste("Trial Store", trial_store),
                             paste("Control Store", control_store)))

# Generate the line chart
ggplot(plot_data, aes(x = monthID_y_m, y = total_sales, color = Store_Type, group = Store_Type)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "Total Sales: Trial vs Control Store (Pre-Trial)",
       x = "Month",
       y = "Total Sales ($)",
       color = "Store") +
  # Expand the y-axis to start at 0 to prevent visual distortion of the gap
  scale_y_continuous(limits = c(0, NA))

```



The store with the 2nd highest score was usually the one which also performed best in plotting test and selected as the control store since it is most similar to the trial store. # Findings for most similar control store for the trial stores are as follows:

Most correlated store for Trial store 77 = 233

Most correlated store for Trial store 86 = 155

Most correlated store for Trial store 88 = 237

Conducting visual checks on customer count trends by comparing the trial store to the respective control stores and other stores

For store 77 vs 233

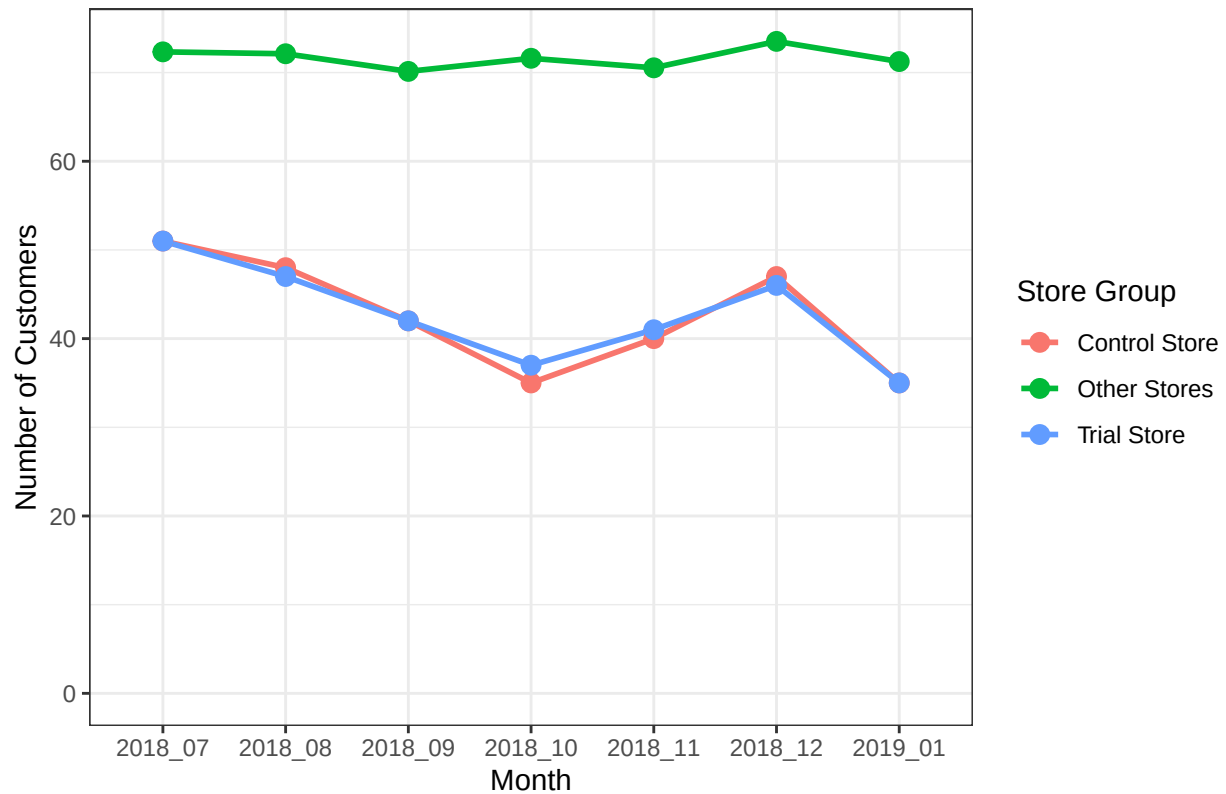
```
library(ggplot2)
library(dplyr)
library(tidyr)

trial_store <- 77
control_store <- 233

# Separating stores into categories and calculate monthly averages
plot_data_cust <- pre_trial_stores_7month %>%
  mutate(Store_Type = case_when(
    STORE_NBR == trial_store ~ "Trial Store",
    STORE_NBR == control_store ~ "Control Store",
    TRUE ~ "Other Stores"
  )) %>%
  # Group by month and the new store type to aggregate the data
  group_by(monthID_y_m, Store_Type) %>%
  summarise(avg_customers = mean(num_customers, na.rm = TRUE), .groups = "drop")
# Create the trend line chart
ggplot(plot_data_cust, aes(x = monthID_y_m, y = avg_customers, color = Store_Type, group = Store_Type)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(
    title = "Total Customer Counts: Trial vs Control vs Other Stores (Pre-Trial)",
    x = "Month",
    y = "Number of Customers",
    color = "Store Group"
```

```
) +  
  scale_y_continuous(limits = c(0, NA))
```

Total Customer Counts: Trial vs Control vs Other Stores (Pre-Trial)



For store 86 vs 155

```
library(ggplot2)  
library(dplyr)  
library(tidyr)
```

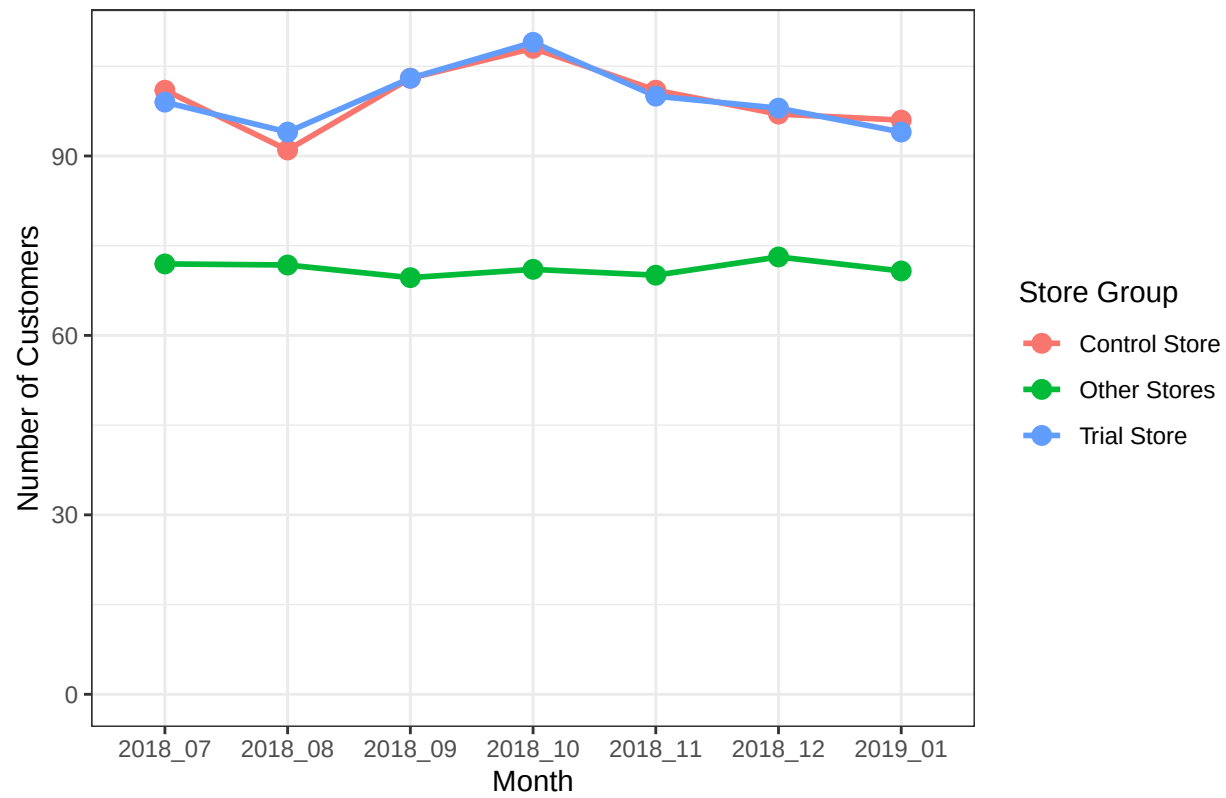
```

trial_store <- 86
control_store <- 155

# Separating stores into categories and calculate monthly averages
plot_data_cust <- pre_trial_stores_7month %>%
  mutate(Store_Type = case_when(
    STORE_NBR == trial_store ~ "Trial Store",
    STORE_NBR == control_store ~ "Control Store",
    TRUE ~ "Other Stores"
  )) %>%
  # Group by month and the new store type to aggregate the data
  group_by(monthID_y_m, Store_Type) %>%
  summarise(avg_customers = mean(num_customers, na.rm = TRUE), .groups = "drop")
# Create the trend line chart
ggplot(plot_data_cust, aes(x = monthID_y_m, y = avg_customers, color = Store_Type, group = Store_Type)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(
    title = "Total Customer Counts: Trial vs Control vs Other Stores (Pre-Trial)",
    x = "Month",
    y = "Number of Customers",
    color = "Store Group"
  ) +
  scale_y_continuous(limits = c(0, NA))

```

Total Customer Counts: Trial vs Control vs Other Stores (Pre-Trial)



For store 88 vs 237

```
library(ggplot2)
library(dplyr)
library(tidyr)

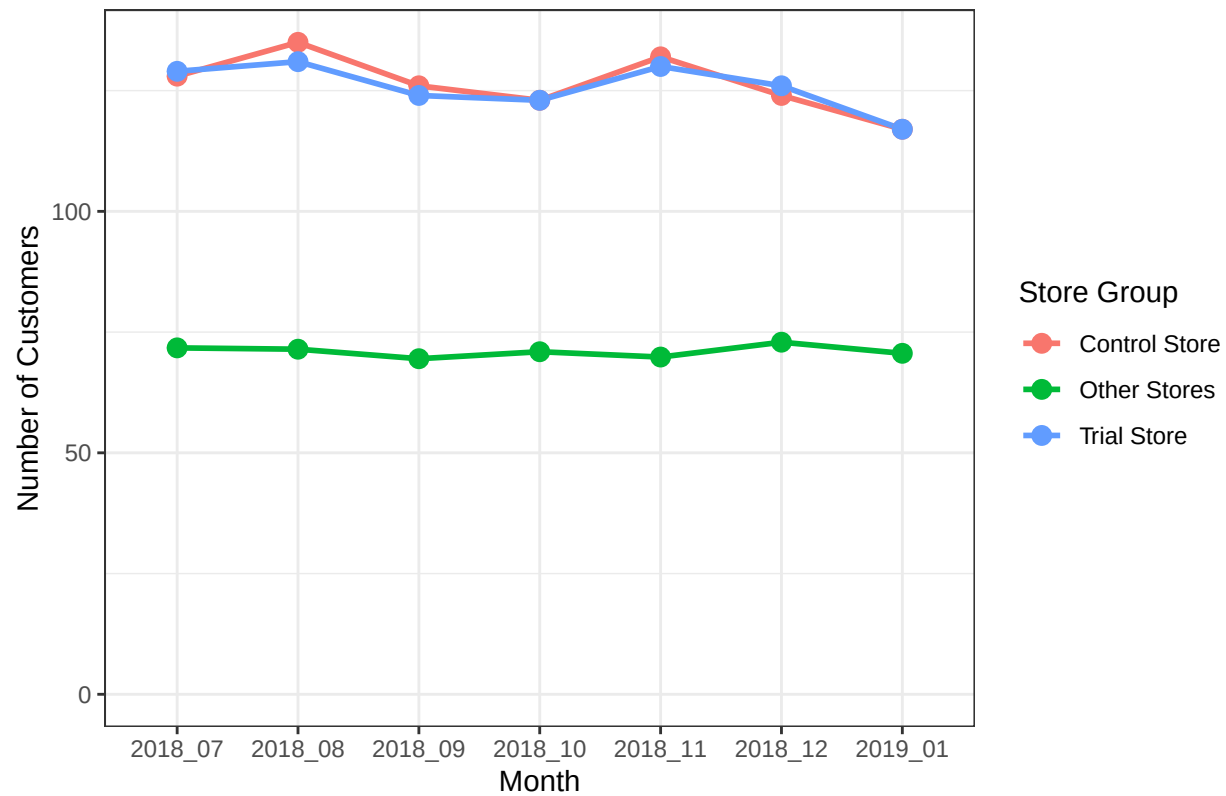
trial_store <- 88
control_store <- 237
```

```

# Separating stores into categories and calculate monthly averages
plot_data_cust <- pre_trial_stores_7month %>%
  mutate(Store_Type = case_when(
    STORE_NBR == trial_store ~ "Trial Store",
    STORE_NBR == control_store ~ "Control Store",
    TRUE ~ "Other Stores"
  )) %>%
  # Group by month and the new store type to aggregate the data
  group_by(monthID_y_m, Store_Type) %>%
  summarise(avg_customers = mean(num_customers, na.rm = TRUE), .groups = "drop")
# Create the trend line chart
ggplot(plot_data_cust, aes(x = monthID_y_m, y = avg_customers, color = Store_Type, group = Store_Type)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(
    title = "Total Customer Counts: Trial vs Control vs Other Stores (Pre-Trial)",
    x = "Month",
    y = "Number of Customers",
    color = "Store Group"
  ) +
  scale_y_continuous(limits = c(0, NA))

```

Total Customer Counts: Trial vs Control vs Other Stores (Pre-Trial)



Assessment of trial From period beyond February 2019.

```
library(dplyr)
# store 77:

trial_store <- 77
control_store <- 233
```

```

# Calculate the sum of pre-trial sales for the Trial Store
pre_trial_sales_trial <- pre_trial_stores_7month %>%
  filter(STORE_NBR == trial_store) %>%
  pull(total_sales) %>%
  sum()
# Calculate the sum of pre-trial sales for the Control Store
pre_trial_sales_control <- pre_trial_stores_7month %>%
  filter(STORE_NBR == control_store) %>%
  pull(total_sales) %>%
  sum()
# Calculate the Scaling Factor
scaling_factor_sales <- pre_trial_sales_trial / pre_trial_sales_control
# Print the factor to see how much we are adjusting by
print(paste("Sales Scaling Factor:", round(scaling_factor_sales, 4)))

```

We'll start with scaling the control store's sales to a level similar to control for any differences between the two stores outside of the trial period.

```
## [1] "Sales Scaling Factor: 1.0236"
```

```

# Apply the scaling factor to the Control Store's FULL dataset
scaled_control_data <- store_monthly_metrics %>%
  filter(STORE_NBR == control_store) %>%
  mutate(control_sales_scaled = total_sales * scaling_factor_sales)

head(scaled_control_data %>% select(monthID_y_m, total_sales, control_sales_scaled))

```

```

## # A tibble: 6 x 3
##   monthID_y_m total_sales control_sales_scaled
##   <chr>         <dbl>         <dbl>
## 1 2018_07         291.           298.
## 2 2018_08         286.           293.
## 3 2018_09         229.           234.
## 4 2018_10         186.           190.
## 5 2018_11         212.           217.
## 6 2018_12         280.           286.

```

```

library(dplyr)
# store 86:

trial_store <- 86
control_store <- 155

# Calculate the sum of pre-trial sales for the Trial Store
pre_trial_sales_trial <- pre_trial_stores_7month %>%
  filter(STORE_NBR == trial_store) %>%
  pull(total_sales) %>%
  sum()
# Calculate the sum of pre-trial sales for the Control Store
pre_trial_sales_control <- pre_trial_stores_7month %>%
  filter(STORE_NBR == control_store) %>%
  pull(total_sales) %>%
  sum()
# Calculate the Scaling Factor
scaling_factor_sales <- pre_trial_sales_trial / pre_trial_sales_control
# Print the factor to see how much we are adjusting by
print(paste("Sales Scaling Factor:", round(scaling_factor_sales, 4)))

## [1] "Sales Scaling Factor: 0.9701"

# Apply the scaling factor to the Control Store's FULL dataset
scaled_control_data <- store_monthly_metrics %>%
  filter(STORE_NBR == control_store) %>%
  mutate(control_sales_scaled = total_sales * scaling_factor_sales)

head(scaled_control_data %>% select(monthID_y_m, total_sales, control_sales_scaled))

## # A tibble: 6 x 3
##   monthID_y_m total_sales control_sales_scaled
##   <chr>         <dbl>         <dbl>
## 1 2018_07         925.         897.
## 2 2018_08         783.         759.
## 3 2018_09        1014.         984.

```

```
## 4 2018_10          964.          935.
## 5 2018_11          899.          872.
## 6 2018_12          850.          824.
```

```
library(dplyr)
# store 88:

trial_store <- 88
control_store <- 237

# Calculate the sum of pre-trial sales for the Trial Store
pre_trial_sales_trial <- pre_trial_stores_7month %>%
  filter(STORE_NBR == trial_store) %>%
  pull(total_sales) %>%
  sum()
# Calculate the sum of pre-trial sales for the Control Store
pre_trial_sales_control <- pre_trial_stores_7month %>%
  filter(STORE_NBR == control_store) %>%
  pull(total_sales) %>%
  sum()
# Calculate the Scaling Factor
scaling_factor_sales <- pre_trial_sales_trial / pre_trial_sales_control
# Print the factor to see how much we are adjusting by
print(paste("Sales Scaling Factor:", round(scaling_factor_sales, 4)))
```

```
## [1] "Sales Scaling Factor: 1.0016"
```

```
# Apply the scaling factor to the Control Store's FULL dataset
scaled_control_data <- store_monthly_metrics %>%
  filter(STORE_NBR == control_store) %>%
  mutate(control_sales_scaled = total_sales * scaling_factor_sales)

head(scaled_control_data %>% select(monthID_y_m, total_sales, control_sales_scaled))
```

```
## # A tibble: 6 x 3
##   monthID_y_m total_sales control_sales_scaled
##   <chr>         <dbl>         <dbl>
```

## 1	2018_07	1448.	1451.
## 2	2018_08	1368.	1370.
## 3	2018_09	1322.	1324.
## 4	2018_10	1348.	1350.
## 5	2018_11	1398.	1400.
## 6	2018_12	1265	1267.

Now that we have comparable sales figures for the control store, we can calculate the percentage difference between the scaled control sales and the trial store's sales during the trial period.

Store 77 vs 233 comparison:

```
library(dplyr)
library(tidyr)

trial_store <- 77
control_store <- 233

pre_trial_sales_trial <- pre_trial_stores_7month %>%
  filter(STORE_NBR == trial_store) %>%
  pull(total_sales) %>%
  sum(na.rm = TRUE)

pre_trial_sales_control <- pre_trial_stores_7month %>%
  filter(STORE_NBR == control_store) %>%
  pull(total_sales) %>%
  sum(na.rm = TRUE)

scaling_factor_sales <- pre_trial_sales_trial / pre_trial_sales_control

# TRIAL PERIOD EVALUATION
# Filter the main dataset for the trial period (February 2019 onwards)
trial_period_data <- store_monthly_metrics %>%
  filter(monthID_y_m >= "2019_02" & monthID_y_m <= "2019_07") %>%
  filter(STORE_NBR %in% c(trial_store, control_store))
```

```

# Apply the pre-calculated scaling factor to the control store ONLY
scaled_trial_data <- trial_period_data %>%
  mutate(scaled_sales = ifelse(STORE_NBR == control_store,
                                total_sales * scaling_factor_sales,
                                total_sales))

# Pivot the data using dynamic column names
performance_eval <- scaled_trial_data %>%
  select(monthID_y_m, STORE_NBR, scaled_sales) %>%
  pivot_wider(names_from = STORE_NBR,
              names_prefix = "Store_",
              values_from = scaled_sales)

# Grab the dynamic column names based on your selected stores
trial_col <- paste0("Store_", trial_store)
control_col <- paste0("Store_", control_store)

# Add context, metadata, and clean formatting
performance_detailed <- performance_eval %>%
  mutate(
    # Explicitly add the store numbers back for context
    Trial_Store_NBR = trial_store,
    Control_Store_NBR = control_store,
    Trial_Sales = .[[trial_col]],
    Scaled_Control_Sales = .[[control_col]],

    # Calculate differences
    Sales_Gap_Value = Trial_Sales - Scaled_Control_Sales,
    Percentage_Difference = abs(Sales_Gap_Value) / Scaled_Control_Sales,

    # Add a plain-text conclusion for each month
    Trial_Result = ifelse(Sales_Gap_Value > 0,
                        "Trial Outperformed",
                        "Trial Underperformed")
  ) %>%
  # Select and reorder only the columns we want to see
  select(
    Trial_Month = monthID_y_m,

```

```

Trial_Store_NBR,
Control_Store_NBR,
Trial_Sales,
Scaled_Control_Sales,
Sales_Gap_Value,
Percentage_Difference,
Trial_Result
) %>%
# Format the numbers so they are easy to read
mutate(
  Trial_Sales = round(Trial_Sales, 2),
  Scaled_Control_Sales = round(Scaled_Control_Sales, 2),
  Sales_Gap_Value = round(Sales_Gap_Value, 2),
  Percentage_Difference = paste0(round(Percentage_Difference * 100, 2), "%")
)

# Convert to standard data.frame so the console prints all columns without hiding them
as.data.frame(performance_detailed)

```

```

##   Trial_Month Trial_Store_NBR Control_Store_NBR Trial_Sales
## 1   2019_02             77             233      235.0
## 2   2019_03             77             233      278.5
## 3   2019_04             77             233      263.5
## 4   2019_05             77             233      299.3
## 5   2019_06             77             233      264.7
##   Scaled_Control_Sales Sales_Gap_Value Percentage_Difference
## 1             249.76         -14.76             5.91%
## 2             203.80          74.70            36.65%
## 3             162.35         101.15            62.31%
## 4             352.53         -53.23             15.1%
## 5             226.22          38.48            17.01%
##           Trial_Result
## 1 Trial Underperformed
## 2 Trial Outperformed
## 3 Trial Outperformed
## 4 Trial Underperformed
## 5 Trial Outperformed

```

Store 86 vs 155 comparison:

```
library(dplyr)
library(tidyr)

trial_store <- 86
control_store <- 155

pre_trial_sales_trial <- pre_trial_stores_7month %>%
  filter(STORE_NBR == trial_store) %>%
  pull(total_sales) %>%
  sum(na.rm = TRUE)

pre_trial_sales_control <- pre_trial_stores_7month %>%
  filter(STORE_NBR == control_store) %>%
  pull(total_sales) %>%
  sum(na.rm = TRUE)

scaling_factor_sales <- pre_trial_sales_trial / pre_trial_sales_control

# TRIAL PERIOD EVALUATION
# Filter the main dataset for the trial period (February 2019 onwards)
trial_period_data <- store_monthly_metrics %>%
  filter(monthID_y_m >= "2019_02" & monthID_y_m <= "2019_07") %>%
  filter(STORE_NBR %in% c(trial_store, control_store))

# Apply the pre-calculated scaling factor to the control store ONLY
scaled_trial_data <- trial_period_data %>%
  mutate(scaled_sales = ifelse(STORE_NBR == control_store,
                              total_sales * scaling_factor_sales,
                              total_sales))

# Pivot the data using dynamic column names
performance_eval <- scaled_trial_data %>%
  select(monthID_y_m, STORE_NBR, scaled_sales) %>%
  pivot_wider(names_from = STORE_NBR,
```

```

names_prefix = "Store_",
values_from = scaled_sales)

# Grab the dynamic column names based on your selected stores
trial_col <- paste0("Store_", trial_store)
control_col <- paste0("Store_", control_store)

# Add context, metadata, and clean formatting
performance_detailed <- performance_eval %>%
  mutate(
    # Explicitly add the store numbers back for context
    Trial_Store_NBR = trial_store,
    Control_Store_NBR = control_store,
    Trial_Sales = .[[trial_col]],
    Scaled_Control_Sales = .[[control_col]],

    # Calculate differences
    Sales_Gap_Value = Trial_Sales - Scaled_Control_Sales,
    Percentage_Difference = abs(Sales_Gap_Value) / Scaled_Control_Sales,

    # Add a plain-text conclusion for each month
    Trial_Result = ifelse(Sales_Gap_Value > 0,
                        "Trial Outperformed",
                        "Trial Underperformed")
  ) %>%
# Select and reorder only the columns we want to see
select(
  Trial_Month = monthID_y_m,
  Trial_Store_NBR,
  Control_Store_NBR,
  Trial_Sales,
  Scaled_Control_Sales,
  Sales_Gap_Value,
  Percentage_Difference,
  Trial_Result
) %>%
# Format the numbers so they are easy to read
mutate(

```

```

Trial_Sales = round(Trial_Sales, 2),
Scaled_Control_Sales = round(Scaled_Control_Sales, 2),
Sales_Gap_Value = round(Sales_Gap_Value, 2),
Percentage_Difference = paste0(round(Percentage_Difference * 100, 2), "%")
)

# Convert to standard data.frame so the console prints all columns without hiding them
as.data.frame(performance_detailed)

```

```

##   Trial_Month Trial_Store_NBR Control_Store_NBR Trial_Sales
## 1    2019_02             86             155        913.2
## 2    2019_03             86             155       1026.8
## 3    2019_04             86             155        848.2
## 4    2019_05             86             155        889.3
## 5    2019_06             86             155        838.0
##   Scaled_Control_Sales Sales_Gap_Value Percentage_Difference
## 1              864.52           48.68           5.63%
## 2              780.32          246.48          31.59%
## 3              819.32           28.88           3.53%
## 4              895.22           -5.92           0.66%
## 5              831.54            6.46           0.78%
##           Trial_Result
## 1   Trial Outperformed
## 2   Trial Outperformed
## 3   Trial Outperformed
## 4 Trial Underperformed
## 5   Trial Outperformed

```

Store 88 vs 237 comparison:

```

library(dplyr)
library(tidyr)

trial_store <- 88
control_store <- 237

```

```

pre_trial_sales_trial <- pre_trial_stores_7month %>%
  filter(STORE_NBR == trial_store) %>%
  pull(total_sales) %>%
  sum(na.rm = TRUE)

pre_trial_sales_control <- pre_trial_stores_7month %>%
  filter(STORE_NBR == control_store) %>%
  pull(total_sales) %>%
  sum(na.rm = TRUE)

scaling_factor_sales <- pre_trial_sales_trial / pre_trial_sales_control

# TRIAL PERIOD EVALUATION
# Filter the main dataset for the trial period (February 2019 onwards)
trial_period_data <- store_monthly_metrics %>%
  filter(monthID_y_m >= "2019_02" & monthID_y_m <= "2019_07") %>%
  filter(STORE_NBR %in% c(trial_store, control_store))

# Apply the pre-calculated scaling factor to the control store ONLY
scaled_trial_data <- trial_period_data %>%
  mutate(scaled_sales = ifelse(STORE_NBR == control_store,
                              total_sales * scaling_factor_sales,
                              total_sales))

# Pivot the data using dynamic column names
performance_eval <- scaled_trial_data %>%
  select(monthID_y_m, STORE_NBR, scaled_sales) %>%
  pivot_wider(names_from = STORE_NBR,
              names_prefix = "Store_",
              values_from = scaled_sales)

# Grab the dynamic column names based on your selected stores
trial_col <- paste0("Store_", trial_store)
control_col <- paste0("Store_", control_store)

# Add context, metadata, and clean formatting

```

```

performance_detailed <- performance_eval %>%
  mutate(
    # Explicitly add the store numbers back for context
    Trial_Store_NBR = trial_store,
    Control_Store_NBR = control_store,
    Trial_Sales = .[[trial_col]],
    Scaled_Control_Sales = .[[control_col]],

    # Calculate differences
    Sales_Gap_Value = Trial_Sales - Scaled_Control_Sales,
    Percentage_Difference = abs(Sales_Gap_Value) / Scaled_Control_Sales,

    # Add a plain-text conclusion for each month
    Trial_Result = ifelse(Sales_Gap_Value > 0,
                        "Trial Outperformed",
                        "Trial Underperformed")
  ) %>%
  # Select and reorder only the columns we want to see
  select(
    Trial_Month = monthID_y_m,
    Trial_Store_NBR,
    Control_Store_NBR,
    Trial_Sales,
    Scaled_Control_Sales,
    Sales_Gap_Value,
    Percentage_Difference,
    Trial_Result
  ) %>%
  # Format the numbers so they are easy to read
  mutate(
    Trial_Sales = round(Trial_Sales, 2),
    Scaled_Control_Sales = round(Scaled_Control_Sales, 2),
    Sales_Gap_Value = round(Sales_Gap_Value, 2),
    Percentage_Difference = paste0(round(Percentage_Difference * 100, 2), "%")
  )

# Convert to standard data.frame so the console prints all columns without hiding them
as.data.frame(performance_detailed)

```

##	Trial_Month	Trial_Store_NBR	Control_Store_NBR	Trial_Sales
## 1	2019_02	88	237	1370.20
## 2	2019_03	88	237	1477.20
## 3	2019_04	88	237	1439.40
## 4	2019_05	88	237	1308.25
## 5	2019_06	88	237	1354.60

##	Scaled_Control_Sales	Sales_Gap_Value	Percentage_Difference
## 1	1406.99	-36.79	2.61%
## 2	1210.08	267.12	22.07%
## 3	1206.48	232.92	19.31%
## 4	1201.17	107.08	8.91%
## 5	1155.40	199.20	17.24%

##	Trial_Result
## 1	Trial Underperformed
## 2	Trial Outperformed
## 3	Trial Outperformed
## 4	Trial Outperformed
## 5	Trial Outperformed

Conclusion

I have concluded that the following are the most suitable control stores against following trial stores:

Stores 77 -> 233

Stores 86 -> 155

Stores 88 -> 237

The results for trial stores 77 and 88 during the trial period, show significant difference in at least 2 of the 3 trial months but this is not the case for trial store 86. We can check with the client if the implementation of the trial was different in trial store 86, but overall, the trial shows a significant increase in sales.

I have attached my final presentation for the Category Manager in following slides below :-

PRESENTATION

Quantium

Trial VS Control Store Report

Task 2: Assessment of Trial-Control Stores





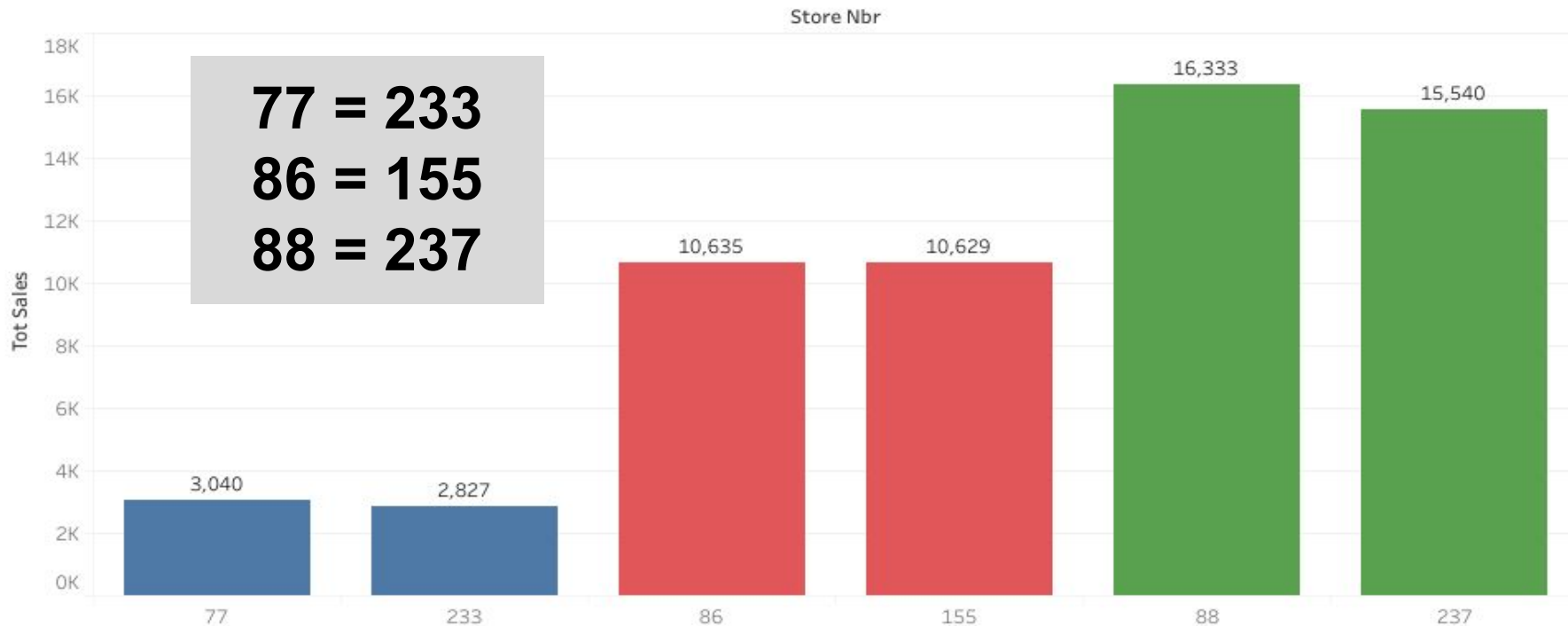
Trial Stores
77, 86, 88





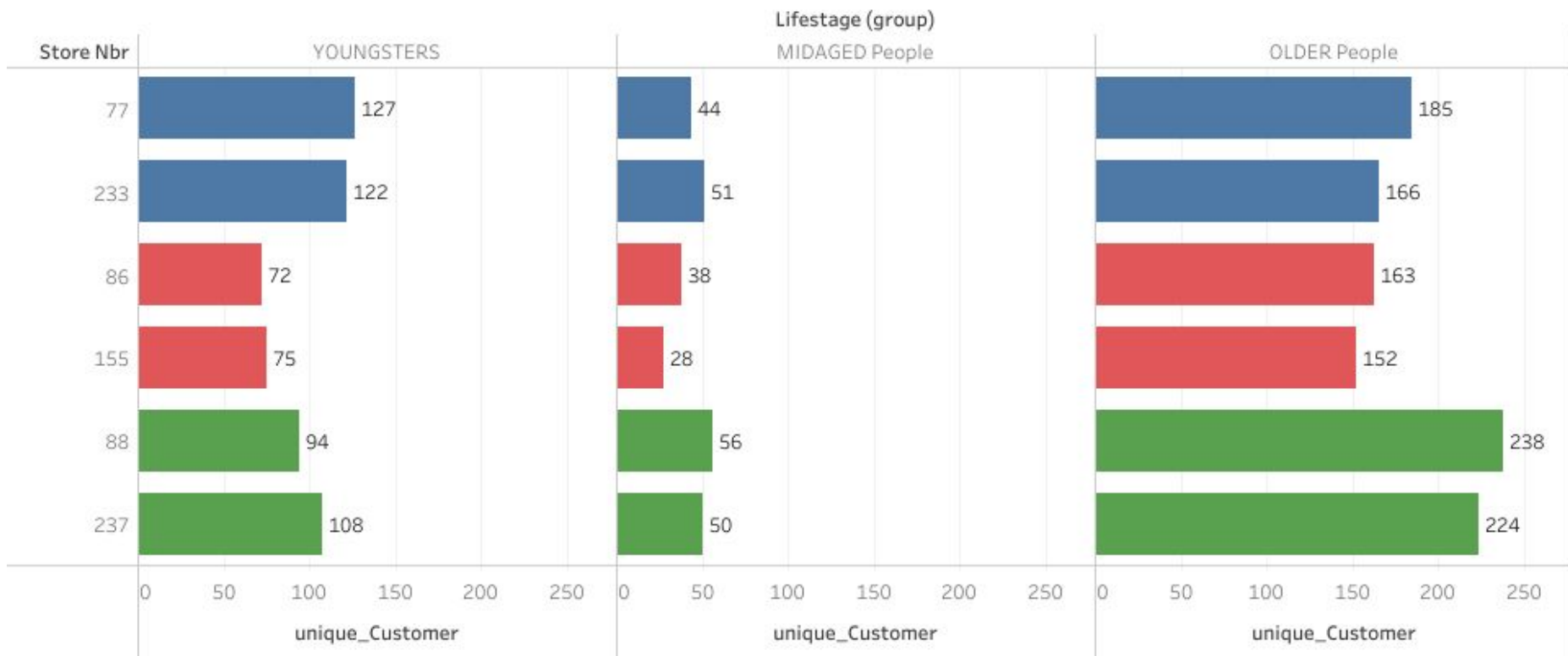
- 04 Sales data comparison
- 05 Customer group breakdown
- 06 Comparison for stores 77 VS 233
- 09 Comparison for stores 86 VS 155
- 12 Comparison for stores 88 VS 237
- 15 Findings

Sales data comparison among respective trial-control stores

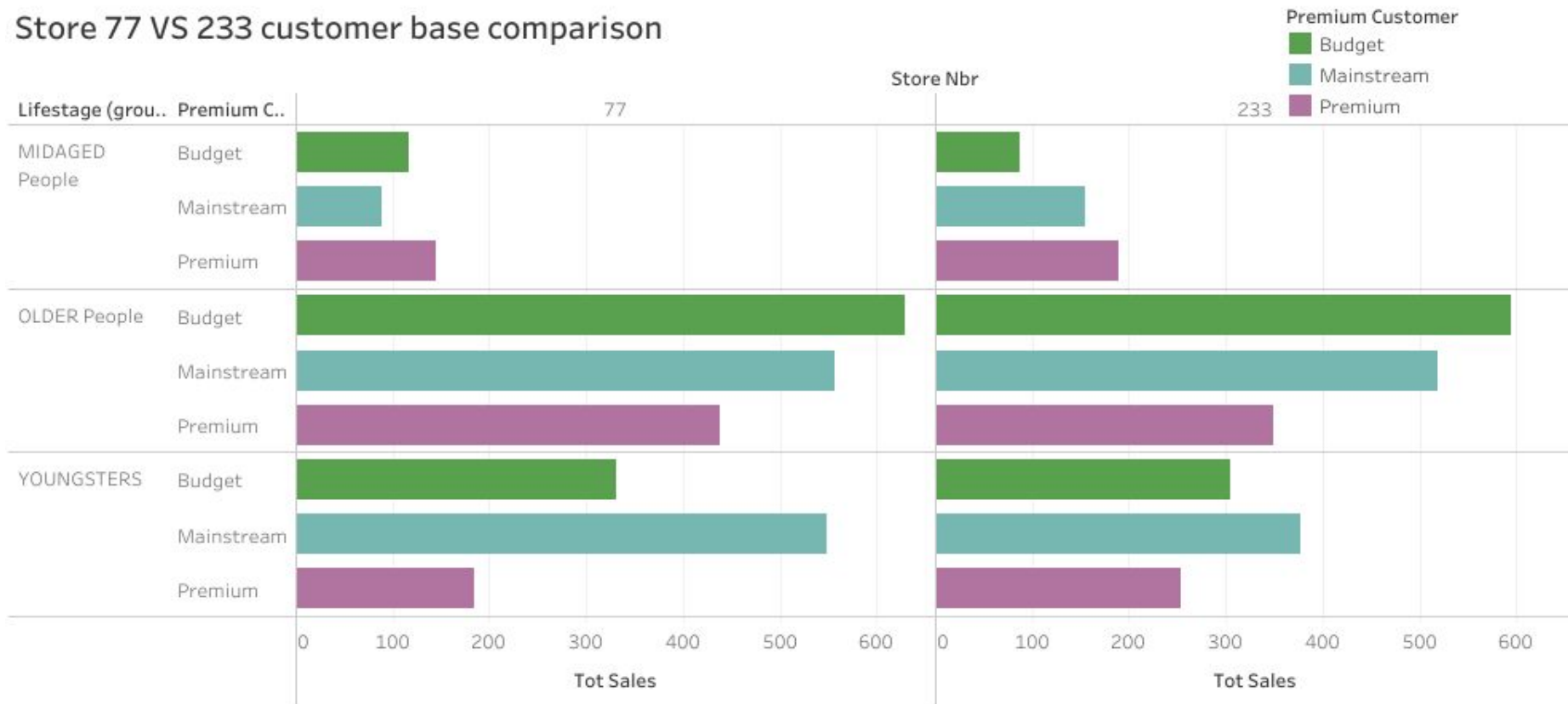


As per the findings the above color coded stores have been the closest match to the trial stores.

Breakdown of customers' diversity by age group in the selected stores

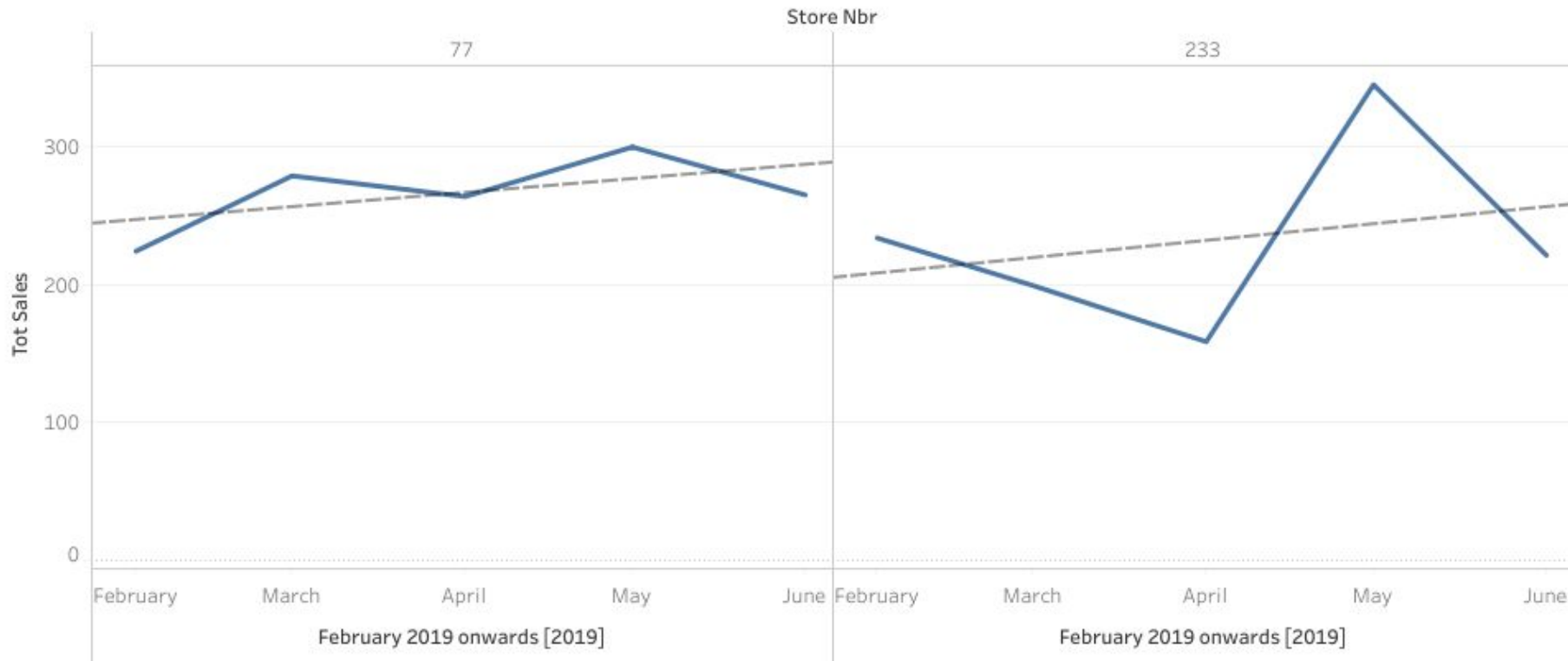


Store 77 VS 233 customer base comparison



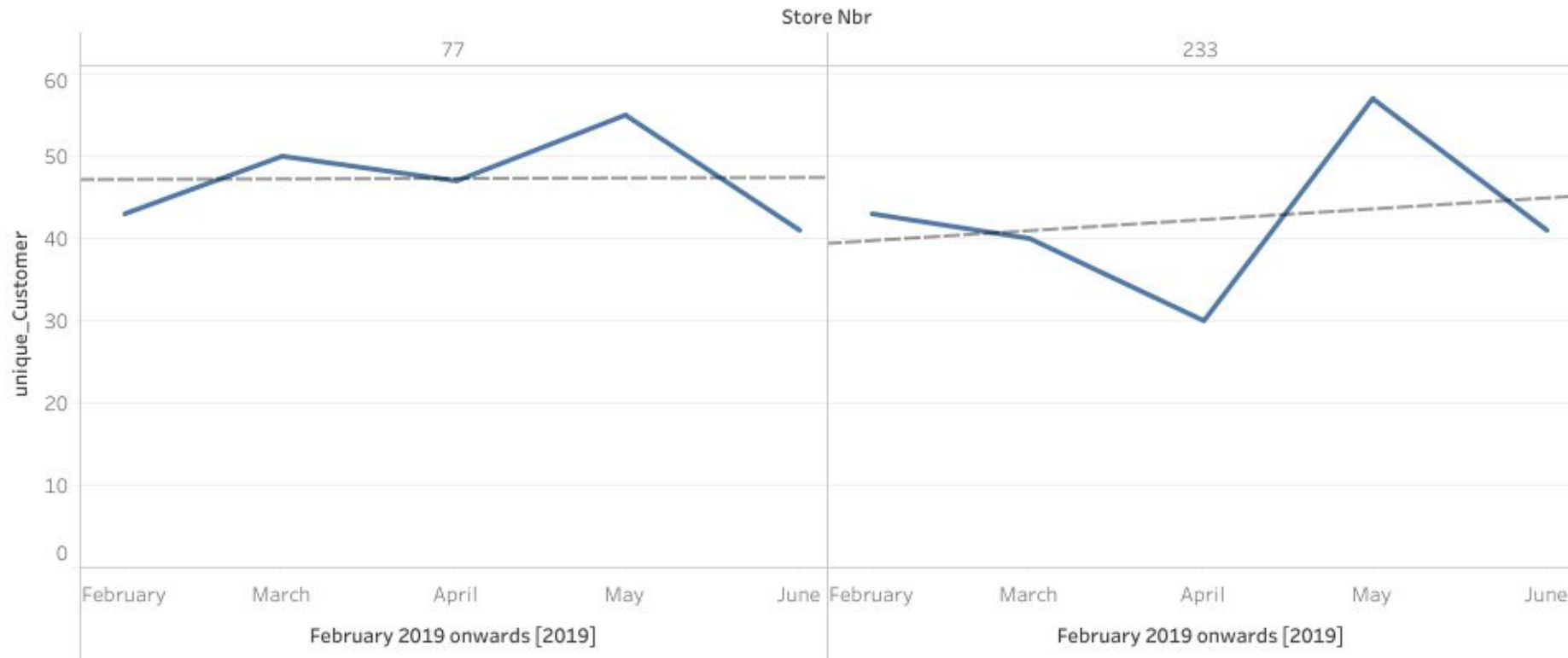
77 VS 233 Sales comparison

Store 77 VS 233 Sales comparison

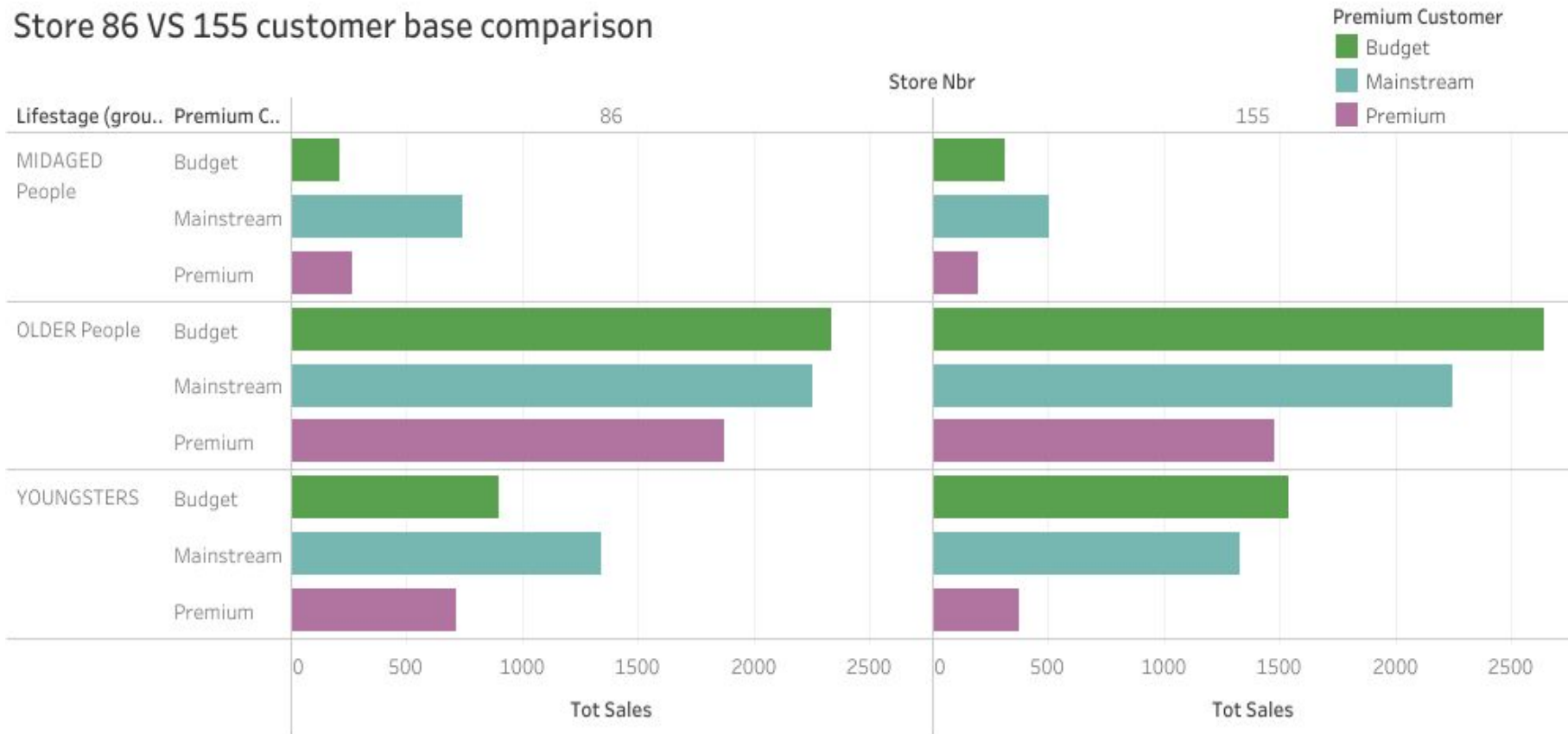


77 VS 233 Customer footfall

Store 77 VS 233 Customer footfall

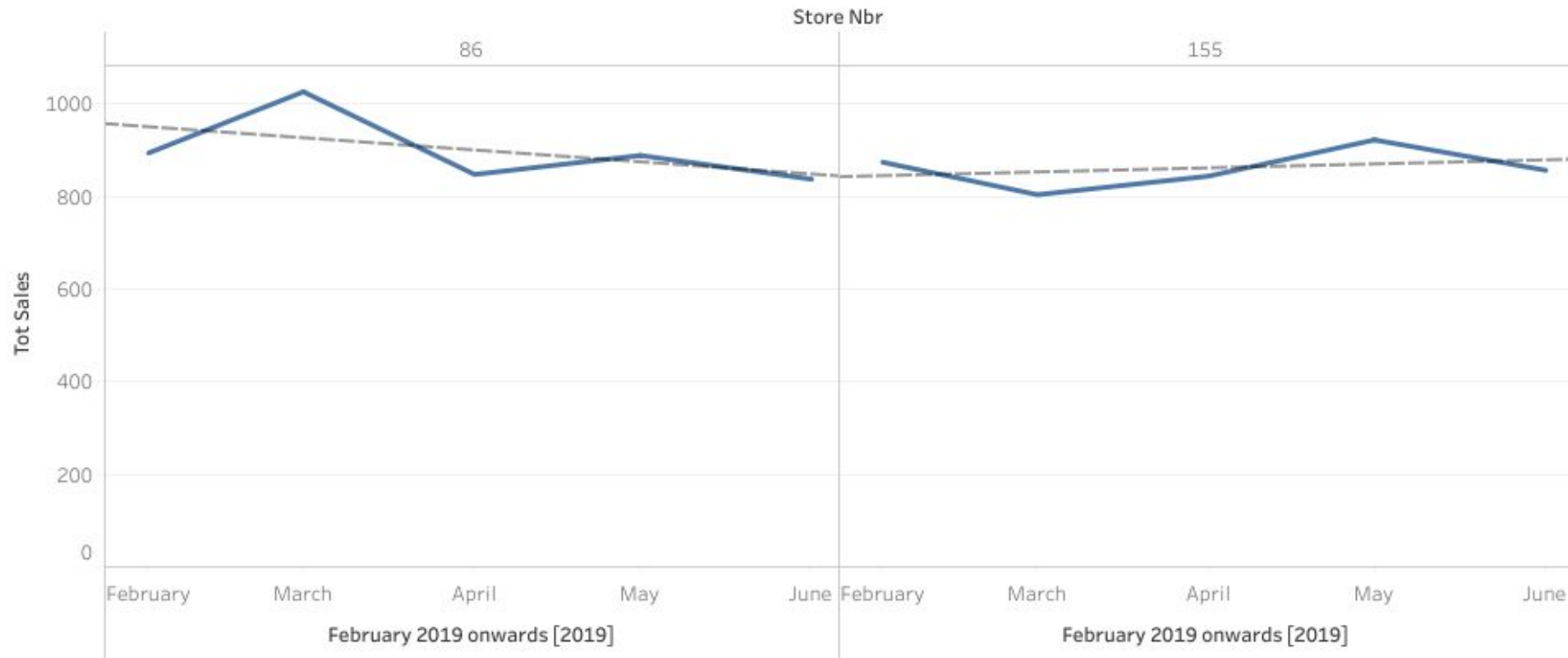


Store 86 VS 155 customer base comparison

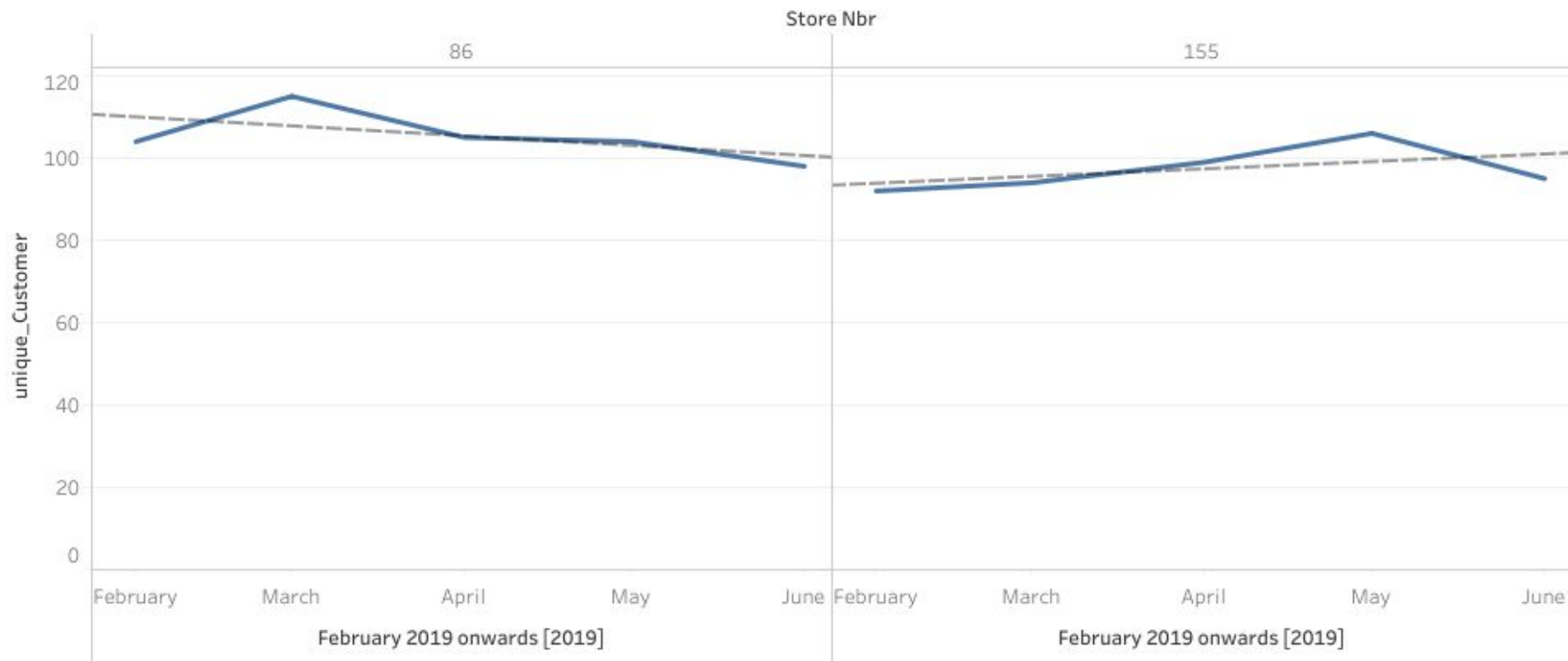


86 VS 155 Sales comparison

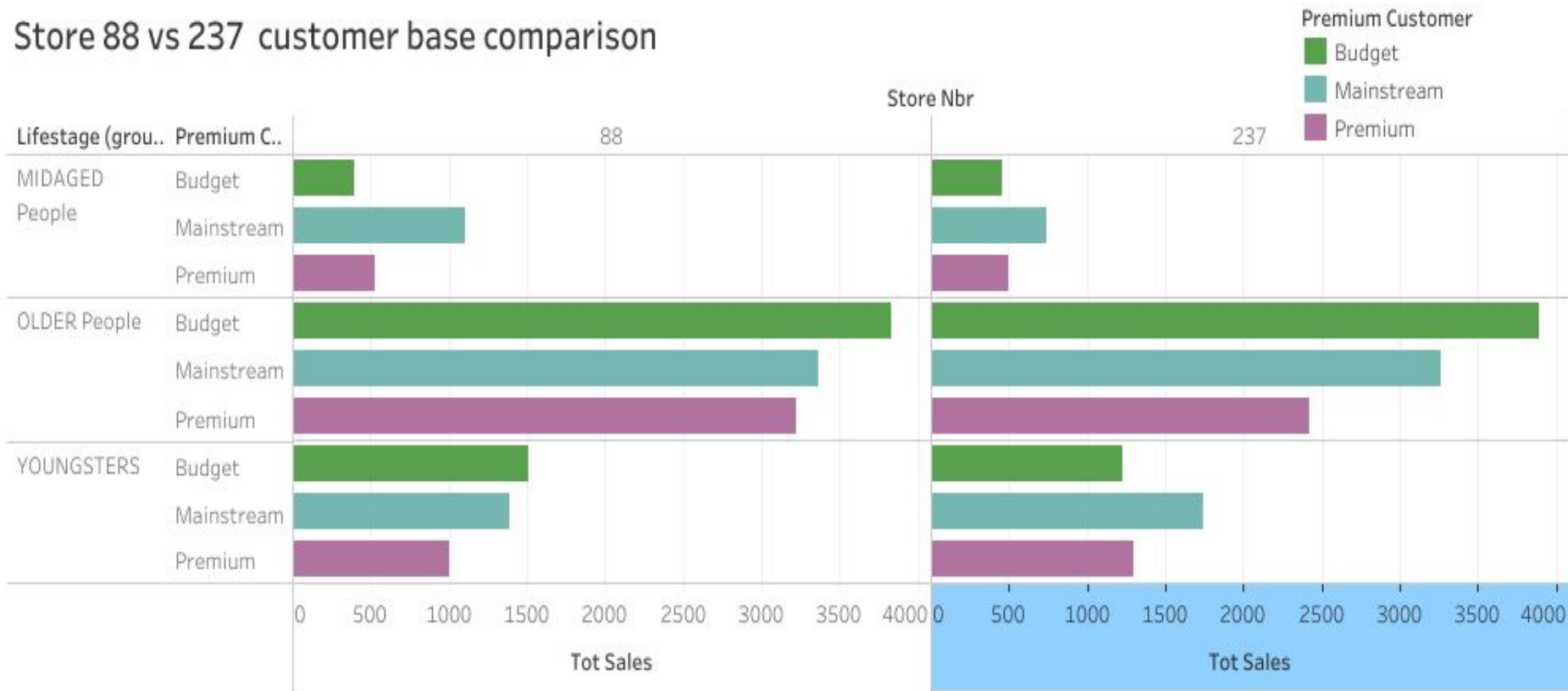
Store 86 VS 155



Store 86 VS 155 Customer footfall

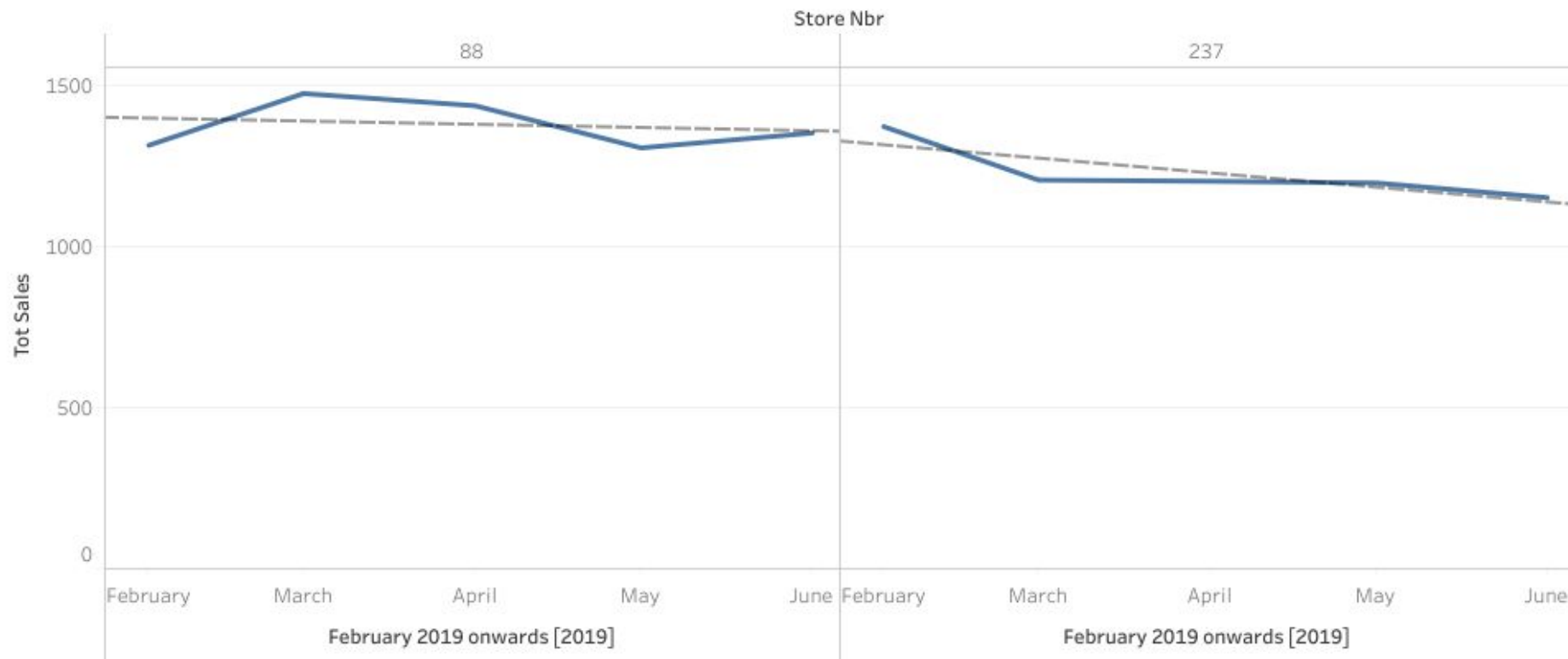


Store 88 vs 237 customer base comparison

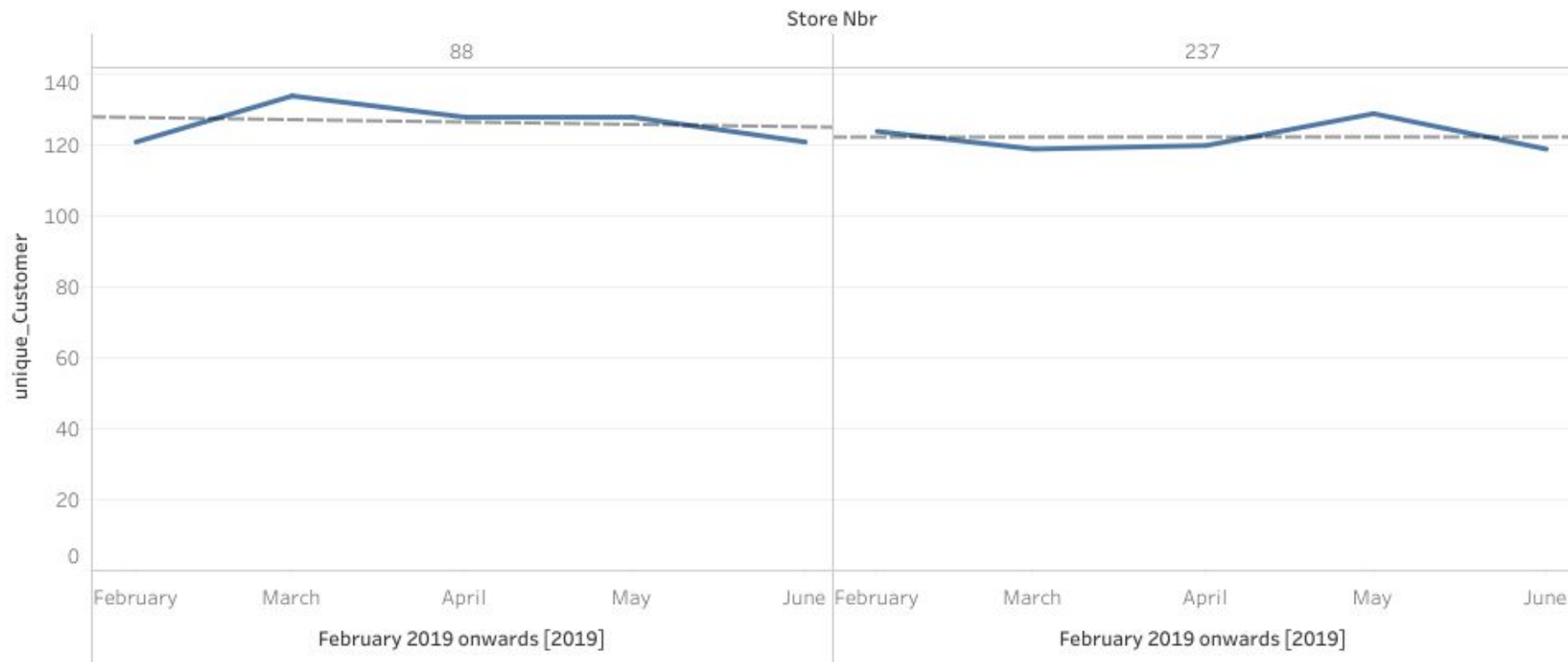


88 VS 237 Sales comparison

Store 88 VS 237 Sales trend line



Store 88 VS 237 Customer footfall



- As per the trend analysis, following control stores are selected against the trial stores for this comparison:
 - **Store 77 -> 233**
 - **Store 86 -> 155**
 - **Store 88 -> 237**
- The results for trial stores 77 and 88 during the trial period, show significant difference in at least 2 of the 3 trial months, whereas this is not the case for store 86.
- overall, the **trial shows a significant increase in sales** compared to control stores with similar stats.

THANK YOU



TUSHAR KOUNDAL
tusharkoundal47@gmail.com